



CS1004 – Object Oriented Programming

Lecture # 12
Tuesday, March 22, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

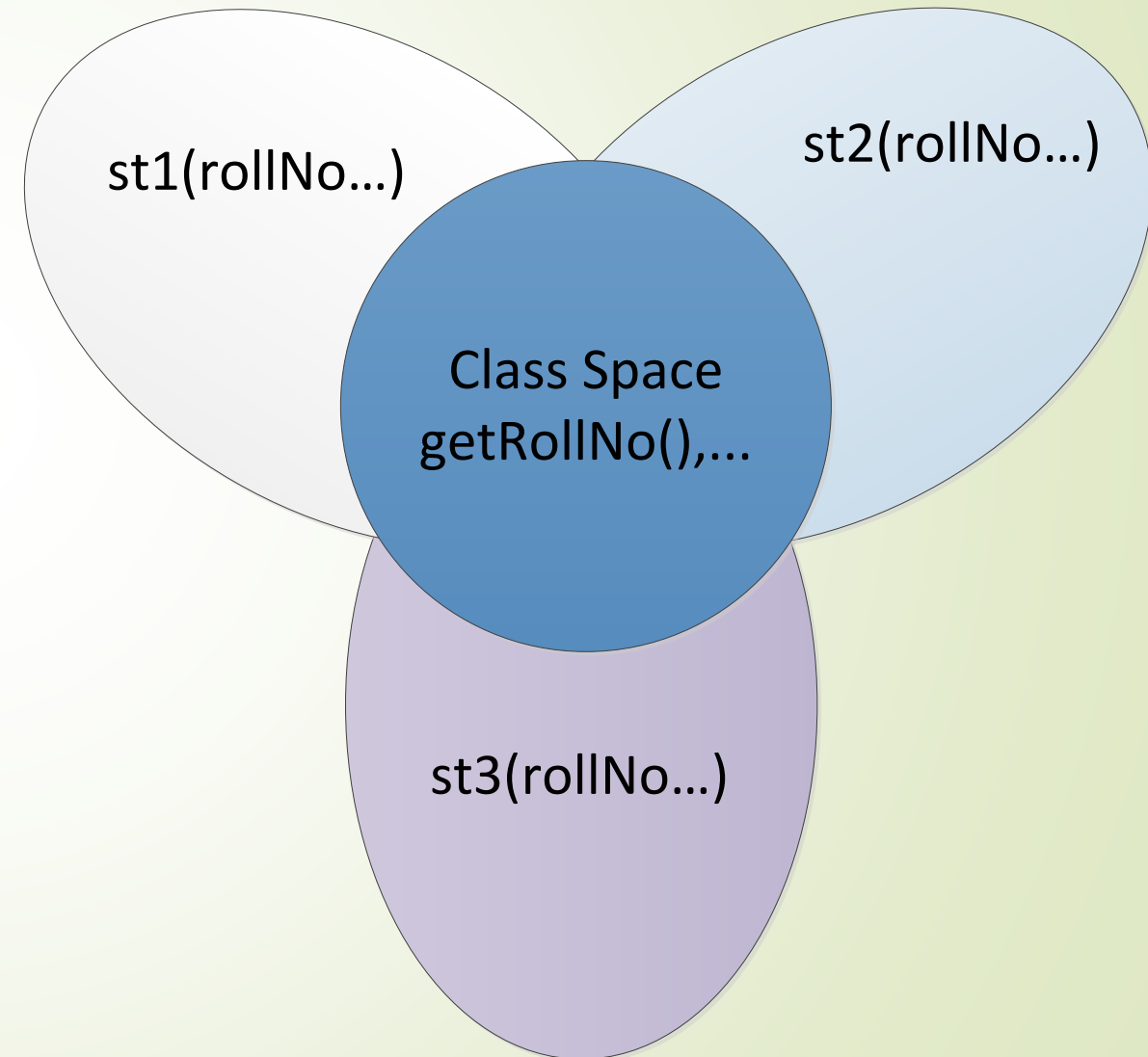
Muhammad Yousaf

```
class Student {  
    int rollNo;  
    char *name;  
    float GPA;  
public:  
    int getRollNo();  
    void setRollNo(int aRollNo);  
    ...  
};
```

3

Constant members

➤ `Student st1, st2, st3;`



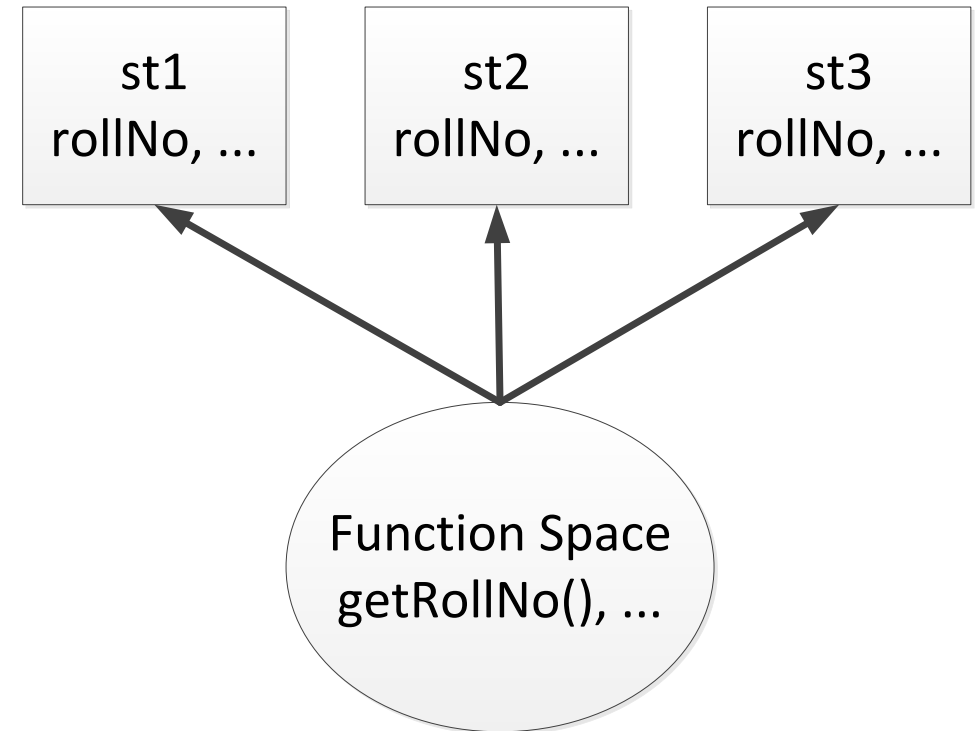
this pointer

- `Student st1, st2, st3;`
- Function space is common for every variable
- Whenever a new object is created:
 - Memory is reserved for variables only
 - Previously defined functions are used over and over again

5

this pointer

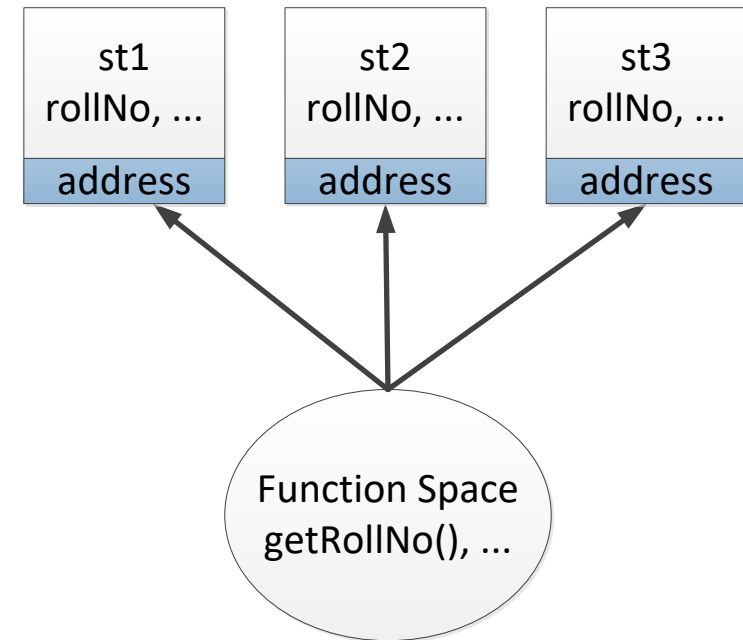
- Memory layout of objects created



- How does the function know on which object to act?

this pointer

- Address of each object is passed to the calling function
- This address is dereferenced by the functions and hence they act on correct objects



- The variable containing the “self-address” is called this pointer

Passing **this** pointer

- ➡ Whenever a function is called '**this**' pointer is passed as a parameter to that function
- ➡ Function with ***n*** parameters is actually called with ***n+1*** parameters
- ➡ There are situations where developer wants to return reference to current object from a function
- ➡ In such cases reference is taken from this pointer like (*this)

Passing **this** pointer (Example)

```
Student& Student::setRollNo(int aNo)
{
    ...
    return *this;
}
Student& Student::setName(char *aName)
{
    ...
    return *this;
}
int main()
{
    Student aStudent;
    Student bStudent;
    bStudent = aStudent.setRollNo(1024);
    ...
    return 0;
}
```


Practice

```
#include<iostream>
using namespace std;
class Test
{
private:
    int x;
    int y;
public:
    Test(int c, int d) { x = c; this->y = d; }
    Test &setX(int a) { x = a; return *this; }
    Test &setY(int b) { y = b; return *this; }
    void print() { cout << "x = " << x << " y = " << y << endl; }
};
int main()
{
    Test obj1(5, 5);
    // Chained function calls. All calls modify the same object
    // as the same object is returned by reference
    obj1.setX(10).setY(20);
    obj1.print();
    return 0;
}
```

const member function

- ➡ There are functions that are meant to be read only
- ➡ Keyword **const** is placed at the end of the parameter list

const member function

- There must exist a mechanism to detect error if such functions accidentally change the data member
- They are just “***read-only***”
- Errors due to typing are also caught at comp

Example

```
class Student {  
    public:  
        int getRollNo() const {  
            return rollNo;  
        }  
};
```

Example

```
bool Student::isRollNo(int aNo) {  
    if (rollNo == aNo) {  
        return true;  
    }  
    return false;  
}
```

Undetected typing mistake

```
bool Student::isRollNo(int aNo) {  
    /*undetected typing mistake*/  
    if (rollNo = aNo) {  
        return true;  
    }  
    return false;  
}
```

With **const** this error is detected by compiler

```
bool Student::isRollNo(int aNo)const {  
    /*compiler error*/  
    if (rollNo = aNo) {  
        return true;  
    }  
    return false;  
}
```

const function

- ➡ Constructor and destructor are used to modify the object to a well defined state
- ➡ Constructors and Destructors cannot be const

const function

- Constant member function cannot change data member
- Constant member function cannot access non-constant member functions
- Constructors and Destructors cannot be **const**

Constant data member

- ➡ Change the class Student such that a student is given a roll number when the object is created and cannot be changed afterwards

Modified Student class

```
class Student {  
    ...  
    const int rollNo;  
public:  
    Student(int aNo);  
    int getRollNo();  
    void setRollNo(int aNo);  
    ...  
};  
Student::Student(int aRollNo)  
{  
    rollNo = aRollNo;  
    /*error: cannot modify a  
    constant data member*/  
}
```

```
void Student::SetRollNo(int i)  
{  
    rollNo = i;  
    /*error: cannot modify a constant  
    data member*/  
}
```

Member initializer list

- A member initializer list is a mechanism to initialize data members
- It is given after closing parenthesis of parameter list of constructor
- In case of more than one member use comma separated list

Class with constant data member

```
class Student {  
    const int rollNo;  
    char *name;  
    float GPA;  
public:  
    Student(int aRollNo):rollNo(aRollNo), name(Null), GPA(0.0) {  
        ...  
    }  
    ...  
};
```

Order of initialization

- ➡ Data member are initialized in order they are declared
- ➡ Order in member initializer list is not significant at all

Example

```
class TEST {  
    int a;  
    int b;  
    int c;  
public:  
    TEST();  
};  
TEST::TEST() :b(10), a(b), c(b)  
{  
}
```

Here

a will have junk value

b will have 10

c will have 10

`const` data member and initializer list

- ➡ Initializer list must be used to initialize a `const` data member

```
Student::Student(int aRollNo) :rollNo(1024)
{
    rollNo = aRollNo;
}
```


const object

- ➡ Objects can be declared constant with the use of **const** keyword
- ➡ Constant objects cannot change their state

const object and non-const member function

```
int main()
{
    const Student aStudent(20);
    int val = aStudent.getRollNo();
    //Error as const object cannot access a non const attribute
    return 0;
}
```

const object

- ➡ `const` objects cannot access “non const” member function
- ➡ Chances of unintentional modification are eliminated

const object

```
class Student {
    int rollNo;
public:
    Student(int aNo):rollNo(aNo){};
    int getRollNo() const;
    void setRollNo(int aNo);
};
int Student::getRollNo() const
{
    return rollNo;
}
int main()
{
    const Student aStudent(20);
    int val = aStudent.getRollNo();
    //Error as const object cannot access a no const attribute
    return 0;
}
```

Questions

